

Jinwon Lee

AI Platform Software · MLOps/Serving · System Software

Website

<https://jinwonlee.github.io/>

This portfolio summarizes platform, infrastructure, serving, observability, and system performance work behind AI applications that need to run reliably in real operating environments. Each project is organized around the problem context, my role, design and operations focus, outcomes, and capability signals.



AI Platform Software

Production AI service operations, Kubernetes platforms, authentication and observability integration



MLOps & Serving

LLM evaluation, model-serving operations, and repeatable GitOps-based delivery flow



System Software

Linux runtime behavior, high-throughput packet processing, and resource-aware performance work

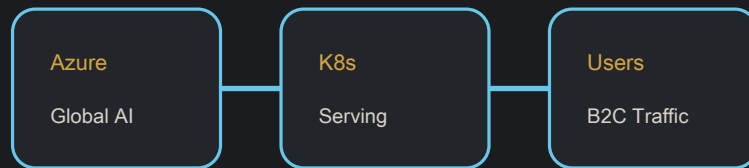


Cloud Infrastructure

Automation with Azure/AWS/OpenShift, Helm/Helmfile, Terraform, and Ansible

Project Details

Problem, role, design/operations focus, outcome, and capability point

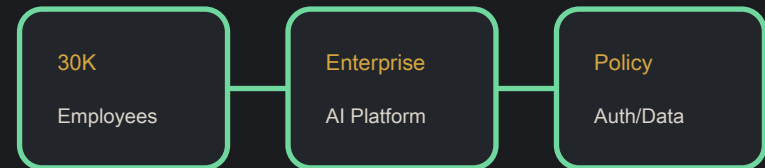


01. Global B2C Generative AI Service

Production generative AI service on Azure with model-serving operations, multi-region readiness, and Kubernetes governance.

- Problem** Prepare a consumer-facing generative AI service for global production rollout.
- Role** Owned platform/SRE design across Azure, Kubernetes operations, model-serving delivery flow, and governance.
- Design/Ops** AI serving efficiency, multi-region deployment, GitOps delivery, network design, and security policy.
- Outcome** Built an operating model for secure and scalable production AI service delivery.
- Capability** Experience designing serving, security, scalability, and multi-region operating structure for production AI services.

Kubernetes Azure AI Serving ArgoCD Python



02. Company-wide AI Chatbot Platform (30K Users)

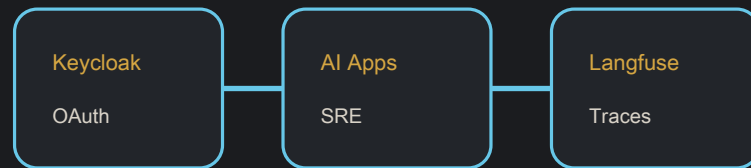
Enterprise AI chatbot platform serving 30,000 employees with Kubernetes, MLOps-style operations, and data protection.

- Problem** Provide a reliable internal AI assistant platform for tens of thousands of employees.
- Role** Led infrastructure, platform, and AI application operations across development, staging, and production clusters.
- Design/Ops** Kubernetes operations, Helm/Helmfile IaC, AI service delivery, authentication, and data protection compliance.
- Outcome** Supported a company-wide AI service with production-grade operating standards.
- Capability** Experience defining Kubernetes-based AI platform operations for tens of thousands of internal users.

Kubernetes Internal Cluster MLOps ArgoCD Helm

Project Details

Problem, role, design/operations focus, outcome, and capability point

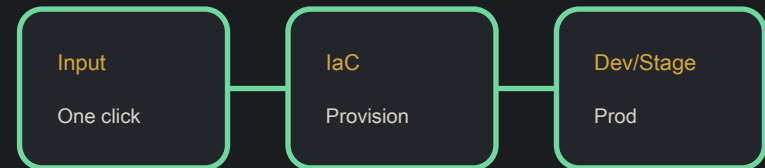


03. Enterprise AI Services Infrastructure

MLOps and SRE operations for browser-integrated AI and Document LLM services with auth and AI observability.

- Problem** Operate multiple internal AI services with enterprise authentication and visibility into AI behavior.
- Role** Handled MLOps/SRE operations and platform integration for AI applications running on Kubernetes-based environments.
- Design/Ops** Keycloak/OAuth authentication, Langfuse AI observability, Kubernetes runtime operations, and Python automation.
- Outcome** Improved operational control for AI services where security, serving health, and runtime visibility matter.
- Capability** Experience connecting LLM application operations with authentication, observability, and runtime reliability.

Kubernetes MLOps Langfuse Keycloak OAuth



04. Kubernetes Cluster Automation

One-click Kubernetes cluster provisioning for repeatable developer and test environments.

- Problem** Cloud developers needed repeatable Kubernetes environments without manual setup overhead.
- Role** Designed and implemented provisioning automation for Kubernetes clusters and shared platform modules.
- Design/Ops** IaC-driven provisioning, Python automation, reusable cluster modules, and developer environment standardization.
- Outcome** Reduced manual operations and improved environment consistency for cloud development workflows.
- Capability** Experience standardizing infrastructure and automating repeatable development and test environments.

Kubernetes Terraform Ansible Python

Project Details

Problem, role, design/operations focus, outcome, and capability point



05. LLM Performance Evaluation Pipeline

MLOps-style evaluation pipeline for LLM quality and performance across Machine Translation tasks.

- Problem** LLM model quality needed repeatable evaluation rather than ad hoc manual checks.
- Role** Built the evaluation workflow and metric tracking path for Machine Translation model comparison.
- Design/Ops** Python evaluation automation, PyTorch-based workloads, MLflow metric tracking, and Kubernetes execution.
- Outcome** Made LLM evaluation results easier to compare and reproduce across model candidates.
- Capability** MLOps experience automating model quality comparison and making evaluation results traceable.

Python

PyTorch

MLflow

MLOps

Kubernetes



06. GitOps CI/CD Pipeline with ArgoCD

GitOps-based delivery pipelines for containerized AI/platform services with Python automation and DevSecOps checks.

- Problem** Containerized AI/platform services needed repeatable, auditable, and secure deployment workflows.
- Role** Designed delivery automation and connected AI service deployment pipelines with security verification steps.
- Design/Ops** ArgoCD GitOps, GitHub Actions, Docker image handling, Kubernetes deployment, and Python glue automation.
- Outcome** Improved deployment consistency and reduced reliance on manual release operations.
- Capability** Experience connecting deployment consistency, security checks, and operations automation for containerized services.

ArgoCD

GitHub Actions

Docker

Kubernetes

Python

Project Details

Problem, role, design/operations focus, outcome, and capability point



07. Centralized Monitoring & Observability Stack

Centralized observability stack for system metrics, container runtime metrics, dashboards, and alerts.

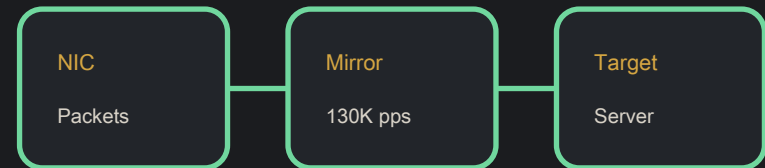
- Problem** Production services needed shared visibility into system behavior and container runtime health.
- Role** Built the monitoring path and dashboards used for platform and service operations.
- Design/Ops** Prometheus metrics, Grafana dashboards, OpenTelemetry signals, Loki logging, and alert rules.
- Outcome** Improved operational visibility for incident detection and runtime analysis.
- Capability** Experience using metrics, logs, and traces for incident detection and runtime analysis.

Grafana

Prometheus

OpenTelemetry

Loki



08. High-throughput Packet Mirroring Application

Real-time packet mirroring application that forwarded up to 130,000 packets/sec under constrained resources.

- Problem** Network/application teams needed real-time packet forwarding under limited system resources.
- Role** Implemented the packet mirroring application and tuned it for resource-constrained operation.
- Design/Ops** High-throughput I/O, Linux runtime behavior, resource limits, and real-time data forwarding.
- Outcome** Delivered a system component capable of forwarding up to 130,000 packets/sec.
- Capability** Experience optimizing Linux runtime behavior and high-throughput data movement under resource constraints.

Linux

Networking

System Software

Performance

Project Details

Problem, role, design/operations focus, outcome, and capability point



09. Distributed CNN Training System

Research on accelerated CNN distributed deep learning with automatic resource-aware layer placement.

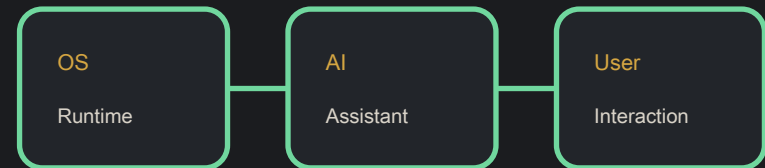
- Problem** Distributed CNN training suffered from network bottlenecks and inefficient layer placement.
- Role** Contributed to research on resource-aware placement for distributed TensorFlow/CNN training.
- Design/Ops** Distributed systems, model parallelism, resource-aware scheduling, TensorFlow, and GPU-aware workloads.
- Outcome** Published research artifacts including USENIX NSDI 2019 poster and arXiv paper; reported up to 2.3x training speedup.
- Capability** Research experience on network bottlenecks, resource placement, and performance improvement for distributed training workloads.

TensorFlow

Distributed Systems

AI Workloads

Python



10. Siri-like AI Assistant on TmaxOS

AI assistant with system-level integration and natural user experience.

- Problem** The OS environment needed an AI assistant that could connect user interaction with system-level capabilities.
- Role** Built AI inference and preprocessing flows and integrated them with TmaxOS-level behavior.
- Design/Ops** Async architecture, system integration, AI model inference, and data preprocessing.
- Outcome** Delivered an AI assistant experience with smoother interaction through asynchronous design.
- Capability** Experience integrating AI functionality with OS-level behavior, asynchronous processing, and user interaction.

Python

AI/ML

TmaxOS

Async Programming

Links and References

Detailed projects and the latest CV are available on the online portfolio.

Website

<https://jinwonlee.github.io/>

GitHub

<https://github.com/JINwonLEE>

LinkedIn

<https://www.linkedin.com/in/jwl1993>

Email

dlwlsdnjsehs1993@gmail.com

Publications

- Alleviating the Network Bottleneck for CNN Distributed Training through Automatic Resource-Aware Layer Placement — USENIX NSDI 2019 (2019)
- Accelerated Training for CNN Distributed Deep Learning through Automatic Resource-Aware Layer Placement — arXiv 2019 (2019)
- Improving Performance of Distributed TensorFlow using CNN Characteristics Exploiting Model Parallelism — ACM EuroSys 2018 (2018)

This document is a portfolio summary based on publicly shareable career and project information. It does not include confidential implementation details or non-public information.